UNIVERSIDAD DE CALDAS

Facultad de Inteligencia Artificial e Ingenierías

Departamento de Sistemas e Informática

Ingeniería de Sistemas y Computación

Análisis y Diseño de Algoritmos

Proyecto Red

Reparto de presupuesto y selección de participantes

Primera entrega: formalización del problema, prueba de NP-dureza y diseño de la solución base

Proyecto semestral 2026-2 · Grupo G2 (Red)

Presenta

Samuel Restrepo Ramírez

samuel.restrepo33481@ucaldas.edu.co

Docente

Luz Enith Guerrero Mendieta

Manizales, Caldas

Noviembre de 2026

1. Introducción

Este informe corresponde a la primera entrega del proyecto semestral del grupo G2 (Red). Contiene la formalización de los dos subproblemas del proyecto, la demostración de que la selección de nodos participantes es un problema NP-difícil mediante una reducción polinomial desde Cobertura Máxima, y el diseño de la solución base que se implementa en el repositorio del proyecto (*https://github.com/samurestrepo/seleccion-submodular-reparto-concavo*), con la deducción de la función de eficiencia de cada algoritmo. Incluye además los resultados con el caso de prueba de la docente (sección 6.9). El código que corresponde a este informe es la versión con la etiqueta primera-entrega del repositorio.

Escenario. Una red de N nodos entrena de forma colaborativa un modelo a lo largo de T rondas sin compartir sus datos crudos. Cada actualización se protege con un mecanismo que agrega ruido calibrado: más ruido protege más, pero reduce la utilidad de la actualización. Cada nodo dispone de un presupuesto limitado de ruido tolerable. En cada ronda se toman dos decisiones, en este orden:

1. **Parte 2, selección:** qué subconjunto de nodos participa en la ronda.
2. **Parte 1, reparto:** cómo se distribuye el presupuesto de la ronda entre los nodos seleccionados.

La numeración de las partes sigue la del enunciado del proyecto. La Parte 1 tiene solución exacta en tiempo polinomial; la Parte 2 es NP-difícil y se aborda con un algoritmo aproximado.

2. Conceptos y notación

Los términos siguientes se usan a lo largo del informe. Se definen aquí para que cada afirmación posterior pueda leerse sin ambigüedad.

Término	Significado en este informe
Tamaño de la entrada	Las cantidades que crecen cuando el problema se hace más grande. Toda función de eficiencia T se expresa en función de ellas.

Ganancia marginal de un nodo (Parte 1)	$\Delta_i(x) = u_i(x+1) - u_i(x)$: lo que aporta al nodo i la unidad de presupuesto número $x+1$.
Función creciente (discreta)	$\Delta_i(x) \geq 0$ para todo x: una unidad más nunca resta utilidad. «Creciente» se usa en sentido amplio (no decreciente): una unidad puede aportar 0.
Función cóncava (discreta)	$\Delta_i(x+1) \leq \Delta_i(x)$: cada unidad aporta a lo sumo lo que aportó la anterior.
2^V	El conjunto de todos los subconjuntos de V. Si $\|V\| = N$, tiene 2^N elementos.
Ganancia marginal de un conjunto (Parte 2)	$\Delta(v \mid S) = f(S \cup \{v\}) - f(S)$: lo que aporta el nodo recién llegado v a un equipo S que ya está formado.
Función monótona	Si $A \subseteq B$, entonces $f(A) \leq f(B)$: agregar nodos nunca baja la utilidad.
Función submodular	Si $A \subseteq B$ y $v \notin B$, entonces $\Delta(v \mid B) \leq \Delta(v \mid A)$: un nodo aporta a un equipo grande a lo sumo lo que aportaría a uno pequeño contenido en él.
Oráculo	Procedimiento que, dado S, devuelve $f(S)$. La función no se escribe como tabla; el costo de un algoritmo se mide en llamadas al oráculo.
Problema de decisión	Problema cuya respuesta es *«sí»* o *«no»*.
Clase P	Problemas de decisión que se resuelven en tiempo polinomial en el tamaño de la entrada.
Clase NP	Problemas de decisión en los que una respuesta *«sí»* se puede verificar en tiempo polinomial si se recibe un certificado de tamaño polinomial (por ejemplo, la solución propuesta).
Reducción polinomial $A \leq_p B$	Transformación, calculable en tiempo polinomial, que convierte cada instancia de A en una instancia de B con la misma respuesta.
NP-difícil	Problema al que se puede reducir en tiempo polinomial cualquier problema de NP. Para probarlo basta reducir hacia él un problema que ya se sabe NP-difícil.

3. Formalización de la Parte 1: reparto óptimo del presupuesto

Los nodos que participan en la ronda ya fueron seleccionados. Se debe repartir entre ellos un presupuesto de B unidades enteras de manera que la utilidad total sea máxima.

Componente	**Definición**
Entrada	$N \in \mathbb{N}$, el número de nodos seleccionados; $B \in \mathbb{N}$, las unidades de presupuesto de la ronda; y para cada nodo $i \in \{1, \ldots, N\}$ una función de utilidad $u_i : \mathbb{N} \to \mathbb{R}$, creciente y cóncava en $\{0, 1, \ldots, B\}$, y un tope $c_i \in \mathbb{N}$, que es lo que le queda al nodo de su presupuesto total.
Variables	$x_1, \ldots, x_N$: unidades asignadas a cada nodo.
Restricciones	$x_i \in \mathbb{Z}$ y $0 \le x_i \le \text{mín}(B, c_i)$ para todo i; además, $x_1 + \ldots + x_N = \text{mín}(B, c_1 + \ldots + c_N)$.
Objetivo	Maximizar la utilidad total $U(x) = u_1(x_1) + \ldots + u_N(x_N)$.
Salida	Un reparto $(x_1, \ldots, x_N)$ que cumple las restricciones y maximiza U, junto con el valor $U(x)$.
Tamaño de la entrada	El par (N, B): el número de nodos y el número de unidades que se reparten, con costo unitario por operación. Cada u_i se recibe como un procedimiento que devuelve $\Delta_i(x)$ en O(1): un coeficiente en las familias log y raíz, o la lista de ganancias en la familia tabla (sección 6.5). *Nota: si B se midiera por el número de bits con que se escribe, el algoritmo sería pseudopolinomial (sección 6.4).*

Observación sobre la restricción de igualdad. El escenario permite gastar «a lo sumo» B unidades. Como cada u_i es creciente, asignar una unidad sobrante nunca reduce la utilidad, de modo que siempre existe un reparto óptimo que usa exactamente B unidades. Por eso las dos versiones tienen el mismo valor óptimo y se trabaja con la igualdad. Con topes, si $c_1 + \ldots + c_N < B$, se reparte esa suma y el resto del presupuesto queda sin usar.

Por qué basta comparar ganancias marginales. La utilidad total es una suma en la que cada término depende solo de lo que recibe su propio nodo (la función es separable); los nodos solo se acoplan a través del presupuesto común. Esta propiedad, junto con la concavidad, es lo que

permite que un algoritmo voraz sea óptimo (sección 6.4).

Relación con el escenario. Las unidades de B son presupuesto de privacidad: dar más a un nodo permite agregar menos ruido a su actualización. En los mecanismos de Laplace y gaussiano, la escala del ruido es proporcional a $1/\varepsilon$, donde ε es el presupuesto gastado, así que cada unidad adicional reduce el error menos que la anterior. Ese rendimiento decreciente es la concavidad de u_i. El tope c_i es lo que le queda al nodo del presupuesto que tiene para todo el proyecto (sección 4.1 del enunciado).

4. Formalización de la Parte 2: selección de nodos participantes

Hay N nodos candidatos y solo k pueden participar en la ronda. La utilidad de un grupo no es la suma de utilidades individuales: depende de cómo se complementan los nodos entre sí, y la mide una función de conjuntos f.

Componente	Definición
Entrada	$N \in \mathbb{N}$ y $k \in \{1, \ldots, N-1\}$; el conjunto de nodos $V = \{1, \ldots, N\}$; y un oráculo $f : 2^V \to \mathbb{R}_{\geq 0}$ monótono, submodular y con $f(\varnothing) = 0$.
Variable	$S \subseteq V$, el conjunto de nodos seleccionados.
Restricción	$\lvert S \rvert \leq k$.
Objetivo	Maximizar $f(S)$.
Salida	Un conjunto S que cumple la restricción, junto con el valor $f(S)$.
Tamaño de la entrada	El par (N, k). Como f se recibe como oráculo, el costo de los algoritmos se mide en número de llamadas a f. Cuando f es una función concreta, el tiempo depende además del tamaño de su descripción: $M = \Sigma \lvert C_v \rvert$ y $\lvert U \rvert$ en la cobertura, y el número de aristas $\lvert E \rvert$ en la cobertura por influencia (sección 6.9).

Observación. El enunciado solo exige que f sea submodular; aquí se añade la monotonía, porque la cota de aproximación la requiere (sección 6.3) y porque las dos funciones que se

usan la cumplen (secciones 5.6 y 6.9). Como f es monótona, agregar nodos a S nunca baja su utilidad; por eso existe un óptimo con $|S| = k$ exactamente, y la restricción $|S| \leq k$ es equivalente a $|S| = k$ en cuanto al valor óptimo.

Relación con el escenario. f mide cuánto aporta el equipo al modelo en conjunto. En la cobertura, los elementos de U representan, por ejemplo, los tipos de dato que aporta cada nodo, y un nodo aporta menos a un equipo que ya cubre la mayoría de esos tipos: esa es la submodularidad. Con el caso de la docente se usa la cobertura por influencia (sección 6.9). La ronda se resuelve en dos pasos, como lo fija el enunciado: primero la Parte 2 y después la Parte 1 sobre los nodos elegidos. Cada paso tiene su garantía, pero no se afirma que la ronda completa optimice un objetivo conjunto.

5. La Parte 2 es NP-difícil

5.1 Estrategia de la demostración

Para probar que un problema es NP-difícil no se analiza un algoritmo concreto: se toma un problema que ya se sabe NP-difícil y se muestra que cualquiera de sus instancias se puede traducir, en tiempo polinomial, a una instancia del problema propio con la misma respuesta. La dirección es **del problema difícil conocido hacia el problema propio**: si la Parte 2 se pudiera resolver en tiempo polinomial, entonces el problema conocido también, lo cual no se espera.

Las reducciones comparan respuestas «sí» o «no», así que se trabaja con versiones de decisión. Esto no pierde generalidad: un algoritmo que resuelva la Parte 2 como problema de optimización encuentra un S^* óptimo, y basta comparar $f(S^*) \geq t$ para responder la versión de decisión. Si la versión de decisión es NP-difícil, la de optimización también lo es.

La demostración se organiza en cinco pasos:

Paso	Contenido
1	Plantear el problema de origen, Cobertura Máxima, como problema de decisión.
2	Construir, a partir de una instancia de Cobertura Máxima, una instancia de la Parte 2.

3	Mostrar que la construcción toma tiempo polinomial.
4	Mostrar que lo construido es una instancia válida de la Parte 2 (la función es monótona y submodular).
5	Mostrar que las dos instancias tienen la misma respuesta (equivalencia ⇒ y ⇐).

5.2 Cobertura Máxima es NP-difícil

Cobertura de Conjuntos (versión de decisión): dados un universo finito U, una colección $\mathscr{C}$ de subconjuntos de U y un entero k, ¿existen a lo sumo k conjuntos de $\mathscr{C}$ cuya unión es todo U? Este problema es NP-completo (Karp, 1972).

Cobertura de Conjuntos se reduce a Cobertura Máxima dejando la misma colección, con $m = |\mathscr{C}|$, $k' = \min(k, m)$ y $t = |U|$. Si k' conjuntos distintos tienen una unión de al menos $|U|$ elementos, esa unión es todo U; y si a lo sumo k conjuntos cubren U, agregar conjuntos hasta tener k' distintos sigue cubriendo U. La traducción toma tiempo polinomial, así que Cobertura Máxima también es NP-difícil. Este es el resultado conocido que cita el enunciado del proyecto.

5.3 Paso 1: el problema de origen

Cobertura Máxima, versión de decisión (CM).

Componente	Definición
Entrada	Un universo finito U; una colección $\mathscr{C} = \{C_1, \ldots, C_m\}$ con cada $C_i \subseteq U$; y dos enteros k, t.
Pregunta	¿Existen k conjuntos distintos de $\mathscr{C}$ cuya unión tiene al menos t elementos?

Parte 2, versión de decisión (P2-D).

Componente	Definición
Entrada	N y k' con $1 \le k' < N$; $V = \{1, \ldots, N\}$; una función f monótona y submodular, dada por una representación que permite evaluarla en tiempo polinomial (en la reducción, los conjuntos y el procedimiento de la sección 5.5); y un umbral t'.

Pregunta	¿Existe $S \subseteq V$ con $\lvert S\rvert \leq k'$ y $f(S) \geq t'$?

Casos triviales de CM. La Parte 2 exige $1 \leq k' < N$. Las instancias de CM con $k = 0$ o $k \geq m$ se resuelven directamente en tiempo polinomial: si $k = 0$, la unión vacía tiene 0 elementos y la respuesta es «sí» si y solo si $t \leq 0$; si $k > m$ no hay k conjuntos distintos y la respuesta es «no»; si $k = m$ basta calcular la unión de todos los conjuntos y compararla con t. En esos casos la reducción entrega una instancia fija de P2-D con la misma respuesta (por ejemplo, $N = 2$, $k' = 1$, $f(S) = \lvert S\rvert$ y $t' = 1$ para «sí» o $t' = 2$ para «no»). En adelante se supone $1 \leq k < m$.

5.4 Paso 2: construcción de la instancia

Cada conjunto de CM se convierte en un nodo, y la utilidad de un grupo de nodos es el número de elementos que cubren juntos sus conjuntos. La correspondencia es:

Cobertura Máxima	**Parte 2 (P2-D)**
El conjunto C_i	El nodo i
m conjuntos	$N = m$ nodos, $V = \{1, \ldots, m\}$
Los elementos de U	Lo que aporta utilidad
Elegir k conjuntos	Elegir S con $\lvert S\rvert \leq k'$, donde $k' = k$
Unión con al menos t elementos	$f(S) \geq t'$, donde $t' = t$

Formalmente, dada la instancia $(U, \mathscr{C}, k, t)$ de CM, se construye la instancia de P2-D con $N = m$, $V = \{1, \ldots, m\}$, $k' = k$, $t' = t$ y la función de cobertura

$$f(S) = \left| \bigcup_{i \in S} C_i \right|$$

es decir, $f(S)$ es el número de elementos distintos de U que pertenecen a alguno de los conjuntos cuyos nodos están en S. Por convención, $f(\varnothing) = 0$.

Ejemplo. Con $U = \{a, b, c, d, e\}$, $C_1 = \{a, b, c\}$, $C_2 = \{c, d\}$ y $C_3 = \{d, e\}$, se obtienen tres nodos y, por ejemplo, $f(\{1, 3\}) = \lvert\{a, b, c, d, e\}\rvert = 5$ y $f(\{1, 2\}) = \lvert\{a, b, c, d\}\rvert = 4$.

5.5 Paso 3: la construcción toma tiempo polinomial

Una reducción solo transfiere la dificultad si la traducción es barata: si traducir ya costara tiempo exponencial, resolver CM a través de la Parte 2 no diría nada sobre la dificultad de la Parte 2. Se cuenta el costo de construir cada pieza:

Pieza	**Qué se hace**	**Costo**
k' y t'	Copiar dos números	O(1)
$V = \{1, \ldots, m\}$	Un nodo por conjunto	O(m)
Los conjuntos C_i	Guardarlos tal como vienen	O($\Sigma\ |C_i|$)
El oráculo f	Entregar el procedimiento que, dado S, marca en un arreglo de $|U|$ casillas los elementos de cada C_i con $i \in S$ y cuenta las casillas marcadas	Construirlo: O(1). Cada llamada: O($|U| + \Sigma\ |C_i|$) en el peor caso, que es $S = V$

En total, la construcción cuesta O($|U| + m + \Sigma\ |C_i|$), que es lineal en el tamaño de la instancia de CM y, por lo tanto, polinomial. Cada evaluación de f también es polinomial.

Por qué f se entrega como oráculo y no como tabla. Una tabla con el valor de f para cada subconjunto de nodos necesitaría una fila por cada elemento de 2^V, es decir, 2^m filas. Escribirla tomaría tiempo exponencial en el tamaño de la entrada de CM y la reducción dejaría de ser polinomial.

5.6 Paso 4: la instancia construida es válida

La Parte 2 solo admite funciones monótonas y submodulares. Si la función de cobertura no cumpliera estas propiedades, la construcción produciría una instancia de otro problema. Sea cub(S) el conjunto de elementos cubiertos por los nodos de S, de modo que $f(S) = |\text{cub}(S)|$. Para un nodo $v \notin S$:

$$\Delta(v \mid S) = |\ C_v \setminus \text{cub}(S)\ |$$

es decir, el aporte del recién llegado es el número de sus elementos que el equipo todavía no cubría.

Monotonía. $\Delta(v \mid S)$ es la cantidad de elementos de un conjunto, así que $\Delta(v \mid S) \geq 0$ para todo S y todo v. Si $A \subseteq B$, se puede pasar de A a B agregando los nodos de $B \setminus A$ uno por uno, y ninguno de esos pasos disminuye f. Por lo tanto $f(A) \leq f(B)$.

Submodularidad. Sean $A \subseteq B$ y $v \notin B$. Todo lo que cubre A también lo cubre B, es decir, $\text{cub}(A) \subseteq \text{cub}(B)$. Entonces los elementos de C_v que siguen sin cubrir frente a B también siguen sin cubrir frente a A:

$$C_v \setminus \text{cub}(B) \subseteq C_v \setminus \text{cub}(A), \text{ y por lo tanto } \Delta(v \mid B) \leq \Delta(v \mid A).$$

Ejemplo con los conjuntos de la sección 5.4, para el recién llegado $v = 3$, cuyo conjunto es {d, e}:

Equipo	**Elementos ya cubiertos**	**Elementos nuevos que aporta el nodo 3**	**Δ(3 | equipo)**
$A = \varnothing$	ninguno	d, e	2
$B = \{2\}$	c, d	e	1

Como $A \subseteq B$, el aporte al equipo más grande es menor: $1 \leq 2$.

5.7 Paso 5: equivalencia de las respuestas

Se debe mostrar que la instancia de CM responde «sí» si y solo si la instancia construida de P2-D responde «sí». Se prueban las dos direcciones.

(⇒) Si CM responde «sí», P2-D responde «sí». Supongamos que existen $C_{i1}, \ldots, C_{ik}$ cuya unión tiene al menos t elementos. Sea $S = \{i_1, \ldots, i_k\} \subseteq V$. Entonces $|S| = k = k'$, por la construcción del paso 2, y por la definición de f:

$$f(S) = |\, C_{i1} \cup \ldots \cup C_{ik} \,| \geq t = t'.$$

Luego la instancia construida responde «sí».

(⇐) Si P2-D responde «sí», CM responde «sí». Supongamos que existe $S = \{v_1, \ldots, v_r\} \subseteq V$ con $r \leq k'$ y $f(S) \geq t'$. Sea $j = k - r$. Como $k < N = m$, el conjunto $V \setminus S$ tiene $m - r \geq j$ nodos; sea $W = \{w_1, \ldots, w_j\} \subseteq V \setminus S$ cualquiera de ellos (si $r = k$, entonces $W = \varnothing$). Se considera la

colección

$$\mathcal{S} = \{ C_{v1}, \ldots, C_{vr}, C_{w1}, \ldots, C_{wj} \}$$

que tiene exactamente $r + j = k$ conjuntos distintos. Por la definición de f, la unión de $\mathcal{S}$ tiene $f(S \cup W)$ elementos. Como f es monótona (paso 4) y $S \subseteq S \cup W$:

$$| \cup \mathcal{S} | = f(S \cup W) \geq f(S) \geq t' = t.$$

Luego CM responde «sí».

5.8 Conclusión

La construcción del paso 2 es una reducción polinomial CM $\leq_p$ P2-D: se calcula en tiempo polinomial (paso 3), produce instancias válidas (paso 4) y conserva la respuesta (paso 5). Como CM es NP-difícil (sección 5.2), **P2-D es NP-difícil**, y en consecuencia también lo es la Parte 2 como problema de optimización (sección 5.1). Además, cuando f es una función de cobertura dada por sus conjuntos, P2-D está en NP: el certificado es el propio S, y comprobar $|S| \leq k'$ y $f(S) \geq t'$ toma tiempo polinomial. Por lo tanto, en ese caso P2-D es NP-completo.

La consecuencia práctica es que no se conoce un algoritmo exacto de tiempo polinomial para la Parte 2, y no se espera que exista salvo que P = NP. Por eso la solución base usa para la Parte 2 un algoritmo voraz aproximado con garantía demostrada (sección 6.3). El algoritmo exacto de búsqueda exhaustiva, con costo proporcional al número de subconjuntos de tamaño k, corresponde a una entrega posterior según el cronograma.

6. Diseño de la solución base

6.1 Qué hace el programa

Para cada caso de prueba, el programa ejecuta una ronda completa:

1. Lee la instancia desde un archivo, sin que sea necesario modificar el código.
2. **Parte 2:** selecciona k nodos con el algoritmo voraz de selección.
3. **Parte 1:** reparte las B unidades entre los nodos seleccionados con el algoritmo voraz de reparto.
4. Verifica las restricciones de la solución obtenida.
5. Escribe la salida, el tiempo de cada parte y la traza completa: nodos elegidos, ganancias marginales evaluadas, reparto del presupuesto y resultado de cada verificación.

Las dos partes se encadenan: la Parte 1 recibe como entrada los k nodos que eligió la Parte 2. Por eso, dentro de una ronda, el tamaño de la Parte 1 es (k, B) y no (N, B).

6.2 Elección de la técnica para cada parte

Cada parte admite varias estrategias del curso. La tabla resume por qué se elige la técnica voraz en ambas, aunque por razones distintas.

Técnica	**Parte 1: reparto**	**Parte 2: selección**
Fuerza bruta	Hay que probar todos los repartos de B unidades entre N nodos; su número crece de forma combinatoria con N y B. El enunciado la prohíbe para esta parte.	Probar los subconjuntos de tamaño k da la respuesta exacta, pero su número crece de forma exponencial. Es el algoritmo exacto de una entrega posterior.
Dividir y vencerás	No se usa: no hay una división evidente en subproblemas independientes del mismo tipo, porque cuánto recibe una mitad depende de la otra.	Igual: el aporte de un nodo depende de los demás elegidos.

Programación dinámica	Es posible (por nodos y unidades restantes), pero cuesta más que el voraz y no aprovecha la concavidad.	No hay subestructura óptima en general: la mejor elección de k nodos no se arma con mejores elecciones más pequeñas.
Voraz	**Óptimo** gracias a la concavidad de las u_i. Es el que pide el enunciado.	**Aproximado**: garantiza al menos $(1 - 1/e) \approx 63\ \%$ del óptimo porque f es monótona y submodular.

6.3 Parte 2: algoritmo voraz de selección

Idea. Se forma el equipo en k pasos. En cada paso se evalúa, para cada nodo que todavía no está en el equipo, cuánto aportaría si entrara, y se agrega el de mayor aporte. Los empates se rompen a favor del nodo de menor índice, para que la ejecución sea reproducible.

```
SELECCIONAR-NODOS(V, k, f)
 1  S ← ∅ ;  valorS ← f(∅)
 2  para paso ← 1 hasta k hacer
 3      mejor ← NIL ;  mejorΔ ← −∞
 4      para cada v ∈ V \ S, en orden creciente de índice, hacer
 5          Δ ← f(S ∪ {v}) − valorS   # evaluación de ganancia marginal
 6          si Δ > mejorΔ entonces
 7              mejor ← v ;  mejorΔ ← Δ
 8      S ← S ∪ {mejor} ;  valorS ← valorS + mejorΔ
 9      TRAZA(paso, aportes evaluados, mejor, mejorΔ, valorS)
10  devolver S, valorS
```

Implementación con la función de cobertura. Se mantiene el conjunto de elementos ya cubiertos. Así, $\Delta(v \mid S) = |C_v \setminus \text{cub}(S)|$ se calcula recorriendo una sola vez los elementos de C_v, sin recalcular la unión completa.

Ejemplo de ejecución con los nodos de la sección 5.4 y $k = 2$:

Paso	Equipo antes	Δ(1 | S)	Δ(2 | S)	Δ(3 | S)	Elegido	f(S) después
1	∅	3	2	2	1	3
2	{1}	—	1	2	3	5

Función de eficiencia. La operación básica es la evaluación de una ganancia marginal (una

llamada al oráculo, línea 5; la llamada a $f(\varnothing)$ de la línea 1 se cuenta aparte). En el paso número p el equipo tiene $p - 1$ nodos, así que se evalúan $N - (p - 1)$ candidatos. El total de evaluaciones es:

$$T(N, k) = \Sigma_{p=1}{}^{k} (N - p + 1) = k{\cdot}N - k(k-1)/2$$

Para obtener su orden se usa un encierro. Por arriba, $T(N, k) \leq k{\cdot}N$. Por abajo, como $k < N$, se tiene $k(k-1)/2 < k{\cdot}N/2$, y entonces $T(N, k) > k{\cdot}N/2$. Con las dos cotas:

$k{\cdot}N/2 < T(N, k) \leq k{\cdot}N$, por lo tanto $T(N, k) \in \Theta(N{\cdot}k)$ evaluaciones de ganancia marginal.

Casos mejor, peor y promedio. Los dos ciclos recorren siempre todos sus valores: no hay ninguna salida anticipada que dependa de los datos. Por eso el número de evaluaciones es el mismo para toda entrada de tamaño (N, k), y los tres casos coinciden en $\Theta(N{\cdot}k)$ evaluaciones. El tiempo, en cambio, depende de lo que cuesta cada evaluación, que varía con la entrada.

Tiempo con la función de cobertura. Sea $M = \Sigma\, |C_v|$ el total de elementos listados en la entrada. En un paso, evaluar todos los candidatos cuesta a lo sumo $O(M)$ (cada lista se recorre una vez, con búsquedas en tiempo constante esperado en el conjunto de cubiertos), y actualizar los cubiertos cuesta $O(|C_{\text{mejor}}|)$. Además, cada paso recorre los N nodos para saltar los que ya están en S. En total, $O(k{\cdot}(N + M))$ en tiempo esperado (para un peor caso determinista basta numerar los elementos de U de 0 a $|U| - 1$ al leer la entrada, en $O(M)$, y usar un arreglo booleano en lugar del conjunto); si ningún nodo tiene su conjunto vacío, $N \leq M$ y el tiempo queda en $O(k{\cdot}M)$.

Tiempo con la cobertura por influencia. Con listas de adyacencia, cada evaluación cuesta $O(1 + \text{grado}(v))$, y agregar el elegido cuesta lo mismo (sección 6.9). En un paso se evalúan a lo sumo los N nodos, así que el paso cuesta $O(N + |E|)$, donde $|E|$ es el número de aristas, y el total es $O(k{\cdot}(N + |E|))$. Con el grafo completo del caso de la docente, $|E| = N(N - 1)/2$ y el total es $O(k{\cdot}N^2)$.

Espacio. La entrada ocupa $\Theta(N + M)$. Además se guardan el equipo S, $O(k)$, y los elementos cubiertos, $O(|U|)$. El espacio adicional es $O(k + |U|)$. Con la cobertura por influencia, el estado del equipo es un número q_w por nodo, y el espacio adicional es $O(N)$.

Garantía de aproximación. Para una función f monótona, submodular y con $f(\varnothing) = 0$, el algoritmo voraz obtiene un conjunto S con

$$f(S) \geq (1 - 1/e) \cdot \mathrm{OPT} \approx 0{,}632 \cdot \mathrm{OPT}$$

donde OPT es la utilidad del mejor conjunto de tamaño k (Nemhauser, Wolsey y Fisher, 1978). La función de cobertura cumple las tres hipótesis (sección 5.6), y la cobertura por influencia también (sección 6.9). La forma exacta de la cota es $1 - (1 - 1/k)^k$, que nunca es menor que $1 - 1/e$; con $k = 5$ vale ≈ 0,672. El voraz no siempre es óptimo: con $C_1 = \{1, 2, 3, 4\}$, $C_2 = \{1, 2, 5\}$, $C_3 = \{3, 4, 6\}$ y $k = 2$, elige primero el nodo 1, que aporta 4, y después el 2, que aporta 1 (empata con el 3 y gana el menor índice), y obtiene 5; el óptimo es {2, 3}, con 6. La razón 5/6 ≈ 0,83 respeta la cota. Es el caso contraejemplo_seleccion de la batería propia (sección 6.8). Las pruebas automáticas del repositorio ya comprueban la cota en instancias pequeñas, comparando contra fuerza bruta; la verificación experimental sistemática corresponde a una entrega posterior.

6.4 Parte 1: algoritmo voraz de reparto con montículo

Idea. El presupuesto se entrega unidad por unidad. Cada unidad va al nodo cuya siguiente unidad aporta más en ese momento. Para encontrar ese nodo rápido se usa un **montículo de máximos** (cola de prioridad) que guarda, para cada nodo, la ganancia de su siguiente unidad. El montículo tiene un elemento por cada nodo que todavía puede recibir unidades. Los empates se rompen a favor del nodo de menor índice; para eso la clave de cada nodo es la pareja $(\Delta_i, -i)$: ante un empate en Δ, en un montículo de máximos gana el $-i$ mayor, que corresponde al menor índice. Con identificadores de texto, como los del caso de la docente (N01, …, N15), los nodos se ordenan por su identificador y, en las dos partes, el empate lo gana el menor en orden lexicográfico.

```
REPARTIR-PRESUPUESTO(S, B, u, c)
 1  para cada i ∈ S hacer  x[i] ← 0
 2  H ← BUILD-MAX-HEAP con las claves (Δ_i(0), -i), i ∈ S con c_i > 0
 3  para unidad ← 1 hasta B, mientras H no esté vacío, hacer
 4      (δ, -i) ← EXTRAER-MÁXIMO(H)     # empate en δ: gana el menor índice
 5      x[i] ← x[i] + 1
 6      si x[i] < c_i entonces INSERTAR(H, (Δ_i(x[i]), -i))
 7      TRAZA(unidad, i, δ)
 8  devolver x,  Σ u_i(x[i])
```

Aquí $\Delta_i(x) = u_i(x+1) - u_i(x)$ es la ganancia marginal definida en la sección 2. Las líneas 4 y 6 se pueden fusionar en una sola operación: reemplazar la raíz por la nueva pareja y hundirla. La línea 6 puede llegar a pedir $\Delta_i(B)$, es decir, usar $u_i(B+1)$; por eso u_i se define para todo $x \geq 0$ (sección 3). Esa última inserción nunca se extrae, así que no cambia el resultado. Un nodo que llega a su tope c_i no vuelve al montículo; si todos llegan, el ciclo termina antes de repartir las B unidades.

Ejemplo de ejecución con dos nodos, P y Q, y $B = 4$. Las ganancias marginales de sus unidades sucesivas son P: 9, 5, 2, 1 y Q: 7, 6, 1, 0 (ambas listas no aumentan, así que las dos funciones son cóncavas):

Unidad	Siguiente ganancia de P	Siguiente ganancia de Q	Recibe	Utilidad acumulada
1	9	7	P	9
2	5	7	Q	16
3	5	6	Q	22
4	5	1	P	27

El resultado es $x_P = 2$, $x_Q = 2$ con utilidad 27, que es el máximo posible: son las cuatro ganancias más grandes de las dos listas.

Por qué es óptimo. Como cada u_i es cóncava, la lista de ganancias de cada nodo no aumenta. La intuición es que, entonces, tomar siempre la mayor ganancia disponible lleva a quedarse con las B mayores ganancias de todas las listas, y la concavidad garantiza que ese conjunto se puede tomar en orden dentro de cada lista (la unidad 2 de un nodo nunca vale más que su unidad 1). La

demostración formal es un argumento de intercambio: partiendo de un reparto óptimo distinto del voraz, se mueve una unidad de un nodo que recibió más que en el voraz a uno que recibió menos, sin empeorar la utilidad total, hasta llegar al reparto voraz. La redacción completa de esta prueba se presenta en una entrega posterior.

La concavidad es indispensable. Con P: 1, 10 y Q: 3, 0 (P no es cóncava, porque su segunda unidad vale más que la primera) y $B = 2$, el voraz entrega la primera unidad a Q (3) y la segunda a P (1), con utilidad 4. El reparto óptimo es dar las dos unidades a P, con utilidad 11.

Función de eficiencia. La operación básica es la comparación entre claves dentro del montículo. Construir el montículo con BUILD-MAX-HEAP cuesta $O(N)$ (Cormen et al., 2022, cap. 6). El ciclo de las líneas 3 a 7 se ejecuta a lo sumo B veces (exactamente B si los topes alcanzan), y en cada vuelta EXTRAER-MÁXIMO e INSERTAR cuestan $O(\lg N)$ cada una, porque el montículo tiene a lo sumo N elementos y su altura es a lo sumo $\lfloor \lg N \rfloor$. Calcular Δ_i cuesta $O(1)$ (sección 3). Sumando:

$$T(N, B) = O(N) + B \cdot O(\lg N) = O(N + B \lg N)$$

Este es el **peor caso**: en cada vuelta el elemento que se hunde o sube recorre toda la altura del montículo. En el montículo del libro guía, una extracción puede detenerse antes de llegar a una hoja; la implementación, en cambio, usa heapq de Python, cuya extracción baja siempre hasta una hoja antes de reacomodar el elemento. Por eso cada una de las B extracciones cuesta $\Theta(\lg N)$ en toda entrada, y el tiempo de la implementación es $\Theta(N + B(1 + \lg N))$ para toda entrada de tamaño (N, B) en la que ningún tope se activa, sin un mejor caso más barato (el término 1 cubre el caso $N = 1$). Esto es consistente con la cota $O(N + B \lg N)$.

Espacio. El montículo y el vector x ocupan $\Theta(N)$ cada uno. Cada u_i se describe con un coeficiente en las familias log y raíz, así que el espacio total es $\Theta(N)$; con la familia tabla se suman las listas, $\Theta(N + \Sigma L_i)$.

Observación sobre el tamaño de *B*. Siguiendo el enunciado, B se cuenta como parte del tamaño de la entrada (número de unidades a repartir), y con esa medida el algoritmo es polinomial en (N, B). Si B se midiera por el número de bits con que se escribe, el costo crecería exponencialmente

con esa longitud; en ese caso se diría que el algoritmo es pseudopolinomial. El modelo de costo se declara en la sección 3.

6.5 Formato de entrada

La solución base lee dos formatos. El formato propio, que se muestra a continuación, es el de la batería de casos del equipo; el de la docente se describe en la sección 6.9. La lectura de la entrada queda aislada en un solo módulo, así que adaptarla no cambia la lógica de los algoritmos.

```
{
  "k": 2,
  "B": 4,
  "nodos": [
    {"id": 1, "cubre": ["a", "b", "c"], "utilidad": {"tipo": "log",  "a": 9.0}},
    {"id": 2, "cubre": ["c", "d"],      "utilidad": {"tipo": "raiz", "a": 7.0}},
    {"id": 3, "cubre": ["d", "e"],      "utilidad": {"tipo": "log",  "a": 6.0}}
  ]
}
```

- **cubre: los elementos que aporta el nodo**. Define la utilidad conjunta de la Parte 2 como función de cobertura, que es monótona y submodular (sección 5.6).
- **utilidad: la función** u_i de la Parte 1, dada por una familia y un coeficiente a, que debe ser un número finito con a > 0 (si no lo es, el archivo se rechaza por error de forma): «log» para $u(x) = a \cdot \ln(1 + x)$ y «raiz» (sin tilde, como en el archivo) para $u(x) = a \cdot \sqrt{x}$. Ambas son crecientes y cóncavas, y cada $\Delta_i(x)$ se calcula en O(1). Para reproducir los ejemplos de la sección 6.4 se admite además la familia «tabla», que recibe directamente la lista de ganancias de las unidades sucesivas (por ejemplo, 9, 5, 2, 1 para el nodo P); como esta familia no garantiza la concavidad, el programa la comprueba antes de ejecutar el reparto. Después de la última ganancia de la lista, cada unidad adicional vale 0; como las ganancias de la lista no son negativas y no aumentan, completar con ceros conserva la concavidad. Cada nodo puede traer además un campo opcional «tope», que es el c_i de la sección 3.
- Un generador produce instancias sintéticas de tamaño creciente con este mismo formato.

6.6 Verificación de restricciones y traza

Después de cada ejecución el programa comprueba, y reporta en la traza, lo siguiente:

Parte	Verificación
Entrada	$1 \le k < N$; $B \ge 0$; identificadores de nodo únicos; cada u_i creciente y cóncava en $\{0, \ldots, B\}$ (ganancias no negativas y que no aumentan, revisadas solo en las unidades que se pueden entregar: en O(1) para log y raíz, que lo cumplen siempre que a > 0, y en O(min(L, B)) para tabla, con L el largo de la lista); con el formato de la docente, además, cada peso de influencia está en [0, 1].
Parte 2	$S \subseteq V$, sin repetidos y con $\lvert S\rvert \le k$; el valor acumulado coincide con $f(S)$ recalculada desde cero (con una tolerancia de redondeo, porque con la influencia f es real); el número de evaluaciones de la ganancia marginal es exactamente $k \cdot N - k(k-1)/2$.
Parte 1	Cada x_i es un entero con $0 \le x_i \le B$ y $x_i \le c_i$; la suma de los x_i es exactamente $\min(B, \Sigma\, c_i)$; la utilidad acumulada coincide con $\Sigma\, u_i(x_i)$ recalculada; el montículo hace una extracción por unidad entregada y una inserción menos por cada nodo que llegó a su tope.

La traza de cada caso registra: en la Parte 2, por cada paso, la ganancia marginal de cada candidato, el nodo elegido y la utilidad acumulada; en la Parte 1, por cada unidad, el nodo que la recibe y su ganancia marginal; al final, el resultado de cada verificación y el tiempo de cada parte. El tiempo se mide con un reloj de alta resolución alrededor de cada algoritmo, sin incluir la lectura ni la escritura de archivos. Para que el registro de la traza no altere la medición, cada algoritmo se ejecuta 21 veces sin traza, que son las ejecuciones cronometradas, y una vez más con traza, que es la que se reporta. Se informa la mediana de las 21 mediciones, junto con el mínimo y el máximo, y el programa comprueba que todas las ejecuciones producen el mismo resultado. Si alguna restricción de la entrada no se cumple, los algoritmos no se ejecutan y la traza indica cuál falló; el motivo queda en la traza del caso (traza.txt y traza.json), y en resumen.csv el caso aparece con los campos de resultado vacíos y todas_cumplen = False.

6.7 Resumen de eficiencia

Algoritmo	Tamaño	Tiempo	Espacio adicional
Construcción de la reducción (sección 5.5)	$(m, \|U\|, \Sigma\|C_i\|)$	$O(\|U\| + m + \Sigma\|C_i\|)$	—
Parte 2: voraz de selección	(N, k)	$\Theta(N \cdot k)$ llamadas a f en todos los casos; $O(k \cdot (N + M))$ con cobertura; $O(k \cdot (N + \|E\|))$ con influencia	$O(k + \|U\|)$ con cobertura; $O(N)$ con influencia
Parte 1: voraz de reparto	(N, B)	Peor caso $O(N + B \lg N)$; con heapq, $\Theta(N + B(1 + \lg N))$ en toda entrada sin topes activos	$\Theta(N)$
Ronda completa (Parte 2 y luego Parte 1 sobre los k elegidos)	(N, k, B)	$O(k \cdot (N + M) + B(1 + \lg k))$ con cobertura; $O(k \cdot (N + \|E\|) + B(1 + \lg k))$ con influencia	$O(k + \|U\|)$ con cobertura; $O(N)$ con influencia

6.8 Supuestos y limitaciones

- Además del caso de la docente (sección 6.9), la solución base se ejecutó con una batería propia de 14 casos válidos y 3 inválidos: los ejemplos de este informe, el contraejemplo de la selección (sección 6.3), casos de borde ($B = 0$, $k = 1$, $k = N - 1$, un nodo que no cubre nada), empates, un caso en el que el tope de un nodo cambia el reparto y cinco instancias sintéticas con $N = 20$, 40, 80, 160 y 320; los inválidos (una utilidad convexa, $k = N$ e identificadores repetidos) deben rechazarse sin ejecutar los algoritmos. El repositorio describe en casos/CATALOGO.md qué comprueba cada caso, su resultado esperado y el obtenido, y guarda las evidencias de todos.
- La utilidad conjunta de la Parte 2 se modela como cobertura en la batería propia, que es el caso que conecta con la reducción, y como cobertura por influencia en el caso de la docente (sección 6.9). Cambiar de función solo cambia el oráculo.
- Esta entrega no incluye las variantes de la sección 4.4 del enunciado (costos de participación, varias rondas con nodos que fallan y nodos vecinos que se afectan), el

algoritmo exacto de la Parte 2 ni la validación experimental del crecimiento. Se abordan en las entregas siguientes, según el cronograma. Los datos de las variantes que trae el caso de la docente se leen y se reportan como información, sin aplicarlos (sección 6.9).

- Las pruebas de optimalidad del voraz de la Parte 1 y de la cota $(1 - 1/e)$ se presentan aquí como fundamento del diseño; su desarrollo completo corresponde a entregas posteriores.

6.9 Caso de la docente

La docente entregó el caso *entrada_G2_red_15_nodos.json*, con 15 nodos, un grafo completo de 105 aristas con pesos de influencia y los parámetros de tres rondas. Está en *casos/profesora/ del repositorio* y se ejecuta, sin cambiar la lógica de los algoritmos, con el mismo comando que la batería propia:

```
python ejecutar.py casos/profesora --salida resultados/profesora
```

Supuestos de modelado adoptados. El archivo no trae los conjuntos que usa la cobertura ni la forma de las funciones de utilidad. Entonces, se adoptaron los tres supuestos de la tabla siguiente, que se declaran como supuestos porque no se han confirmado con la docente.

Campo del archivo	Uso en la solución base
cupo_participantes_por_ronda_k = 5 y presupuesto_ruido_por_ronda_B = 30	k y B de la ronda.
coeficiente_utilidad	Supuesto 1: la utilidad de cada nodo en la Parte 1 es $a \cdot \ln(1 + x)$, con a el coeficiente (familia log de la sección 6.5).
grafo (peso_influencia) y coeficiente_utilidad	Supuesto 2: la utilidad conjunta de la Parte 2 es la cobertura por influencia que se define a continuación. El grafo también podría corresponder a la variante (c) del enunciado, en la que un nodo afecta a sus vecinos; en la solución base se usa para definir f porque el archivo no trae conjuntos de cobertura.
presupuesto_total_nodo = 18	Tope de cada nodo (sección 3). En una sola ronda, el

	nodo lo puede gastar completo.
numero_rondas = 3, costo_participacion y presupuesto_costos_participacion_por_ronda = 24	Supuesto 3: son datos de las variantes (b) y (a). Se leen y se reportan como información, sin aplicarlos.

Cobertura por influencia. Cada nodo w vale su coeficiente a_w. Si w participa, su información entra completa al modelo; si no, cada nodo elegido v la transmite con probabilidad p_{vw}, el peso de la arista $\{v, w\}$, de forma independiente ($p_{vw} = 0$ si no hay arista, y $p_{ww} = 1$). La probabilidad de que la información de w siga sin entrar es $q_w(S) = \Pi_{v \in S} (1 - p_{vw})$, y la utilidad del equipo es lo que se espera que entre:

$$f(S) = \Sigma_{w \in V} a_w \cdot (1 - q_w(S))$$

Para un nodo $v \notin S$, el aporte del recién llegado es $\Delta(v \mid S) = \Sigma_w a_w \cdot p_{vw} \cdot q_w(S)$: la parte de cada w que todavía falta, multiplicada por la probabilidad de que v la traiga. La función es monótona porque agregar un nodo solo multiplica cada q_w por un número entre 0 y 1, y es submodular porque, si $A \subseteq B$, cada $q_w(B) \leq q_w(A)$, y por lo tanto $\Delta(v \mid B) \leq \Delta(v \mid A)$. Además $f(\varnothing) = 0$, así que se aplica la cota de la sección 6.3. Si todos los pesos valen 0 o 1, f es una cobertura ponderada en la que cada nodo cubre su vecindad cerrada (él mismo y sus vecinos con peso 1); si además todos los a_w valen 1, es la cobertura de la reducción. Con listas de adyacencia, cada evaluación cuesta O(1 + grado(v)) (sección 6.3).

Resultados. La tabla resume la ejecución. La traza completa está en *resultados/profesora/entrada_G2_red_15_nodos/ (salida.json, traza.json y traza.tx*t), y el resumen, en resultados/profesora/resumen.csv.

Dato	**Valor**
Equipo elegido, en orden	N11, N14, N09, N15, N13
Utilidad conjunta f(S)	145,5795, con 65 evaluaciones de ganancia marginal (5 · 15 − 5 · 4/2)

Reparto	N11: 5, N14: 7, N09: 7, N15: 6, N13: 5 (30 unidades; ningún nodo llega a su tope de 18)
Utilidad del reparto U(x)	88,5640, con 30 extracciones y 30 inserciones
Tiempo (mediana de 21 repeticiones)	Parte 2: 0,171 ms; Parte 1: 0,038 ms
Verificaciones	Todas se cumplen: entrada, Parte 2, Parte 1 y ejecución
Datos de las variantes (información)	Costo de participación de S: 23, frente a un presupuesto de 24; 3 rondas

Lectura de la traza. En la Parte 2, la ganancia del nodo elegido baja de 103,49 en el primer paso a 2,35 en el quinto: cada nodo nuevo aporta menos porque el equipo ya representa buena parte de la red, que es la submodularidad en los datos. En la Parte 1, las unidades rotan entre los cinco nodos en orden de ganancia marginal decreciente, y N14, el de mayor coeficiente entre los elegidos (10,1), termina con 7 unidades, empatado con N09 (9,9). Los tiempos cambian un poco de una corrida a otra; los reportados son los de las evidencias de la versión entregada.

Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2022). *Introduction to Algorithms* (4.ª ed.). MIT Press.

Brassard, G. y Bratley, P. (1997). *Fundamentos de algoritmia*. Prentice Hall.

Karp, R. M. (1972). Reducibility among combinatorial problems. En R. E. Miller y J. W. Thatcher (eds.), *Complexity of Computer Computations* (pp. 85–103). Plenum Press.

Nemhauser, G. L., Wolsey, L. A. y Fisher, M. L. (1978). An analysis of approximations for maximizing submodular set functions—I. *Mathematical Programming*, 14, 265–294.

Universidad de Caldas (2026). *Proyecto semestral 2026-2, Análisis y Diseño de Algoritmos*. Enunciado del proyecto.